

Homework #4: CSE 460 Submission Template

A)

Software Architecture You have Identified: Model-View-Presenter (MVP)

Before getting into the justification, I want to be upfront about the assumptions I made when designing this system, because they meaningfully shape the architecture. First, the problem states there is one database server per state, and each restaurant in that state sends its transaction data to that server. Transactions are stored simply as the name of the food item sold and the income from each transaction. I interpret this as meaning there is no mechanism to identify individual restaurant-level data within those servers. Second, the stated functional requirements are strictly read-only: display the ten most popular items sold each day, and display the breakfast, lunch, dinner, and beverage menus. No ordering, no admin management, no content editing. Third, the problem mentions a future mobile app. A mobile app that only mirrors a read-only web experience adds limited value to the customer, so I'm assuming the mobile platform will eventually support mobile ordering at the restaurant level. I'm not designing that system now, but I wanted to use an architecture that would make adding it as painless as possible later.

With those assumptions in place, MVP is the right call. The core reason comes down to what sits between the data and the display. In a standard MVC setup, the Model can communicate directly back to the View after the Controller acts. With 50 separate state database servers, letting any View touch the Model directly means every View implementation has to independently handle that inconsistency. That's duplicated logic and fragility baked in from the start. MVP eliminates that by making the Presenter the only path between Model and View. The Aggregation Scheduler (Model) reads from all 50 state servers, normalizes the transaction data, and writes cleaned results into the Menu Database (also Model). The Presenter then queries that database and hands off display-ready JSON. The Web Application and Mobile Application are both passive Views. They render what the Presenter gives them and nothing else.

From a deployment standpoint, this separation pays off immediately. Both View containers sit on client devices (browser, mobile), the Presenter and Aggregation Scheduler live inside the AWS Cloud Hosting Platform, and the database lives in its own bounded deployment node. Each layer can scale, update, or fail independently. The Aggregation Scheduler runs as a cron job on its own App Server (EC2/Docker), so it doesn't compete with the Presenter for resources during peak web traffic. When

the mobile ordering system eventually gets built, new Model services can be added behind the same Presenter interface - the existing Views don't need to change, and the Presenter just exposes additional endpoints.

From a usability standpoint, the Passive View approach means the web and mobile experiences are always consistent. The Presenter handles all the normalization logic, so a customer using the web app and one using the future mobile app see identically structured data regardless of which state's server their data came from. Testability is also strong as the Presenter can be tested in isolation using mocked Model responses, and the Views can be tested independently of any database. Maintainability holds up long term because each layer has exactly one job: the Model aggregates and stores, the Presenter normalizes and routes, the View renders.

B)

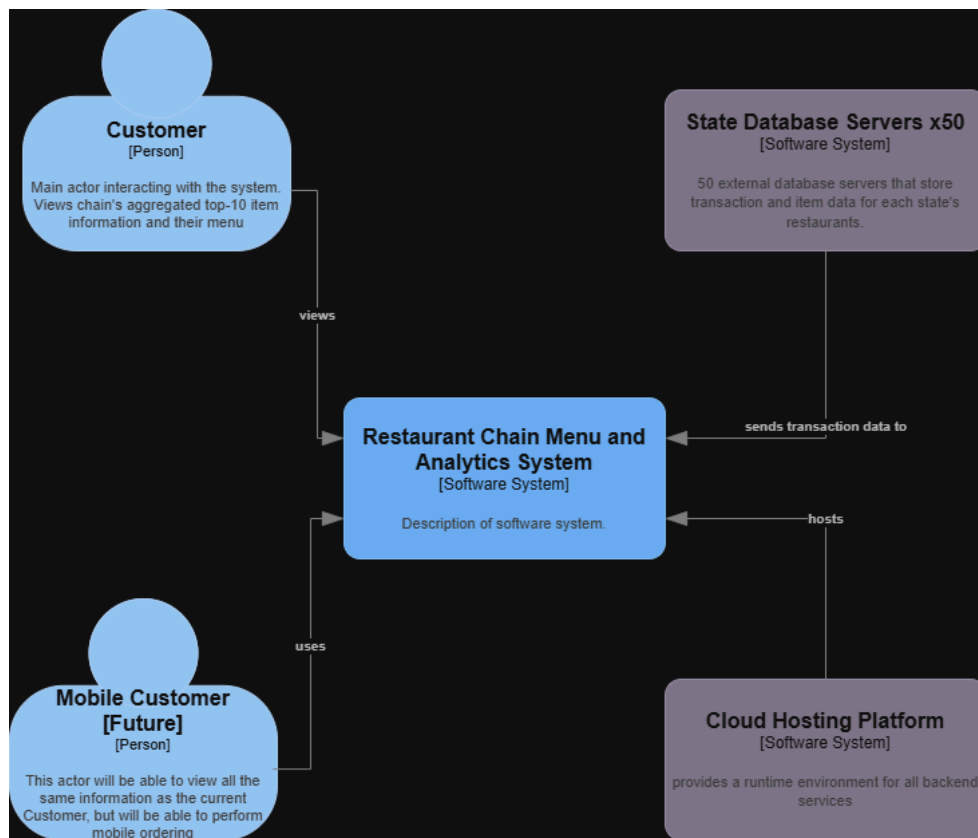


Figure 1. Context Diagram - My context diagram shows the Restaurant Chain Menu and Analytics System as the central software system. There are two human actors: the Customer, who views the chain's aggregated top-10 item information and menus through the system, and the Mobile Customer [Future], who will eventually access the same information plus mobile ordering. Two external software systems connect to it: the State Database Servers $\times 50$, which send transaction data into the system, and the Cloud Hosting Platform, which hosts the system's backend. I included the Mobile Customer as a future actor intentionally.

The system doesn't serve them today, but showing them at this level signals that the architecture needs to support their eventual onboarding without structural rework.

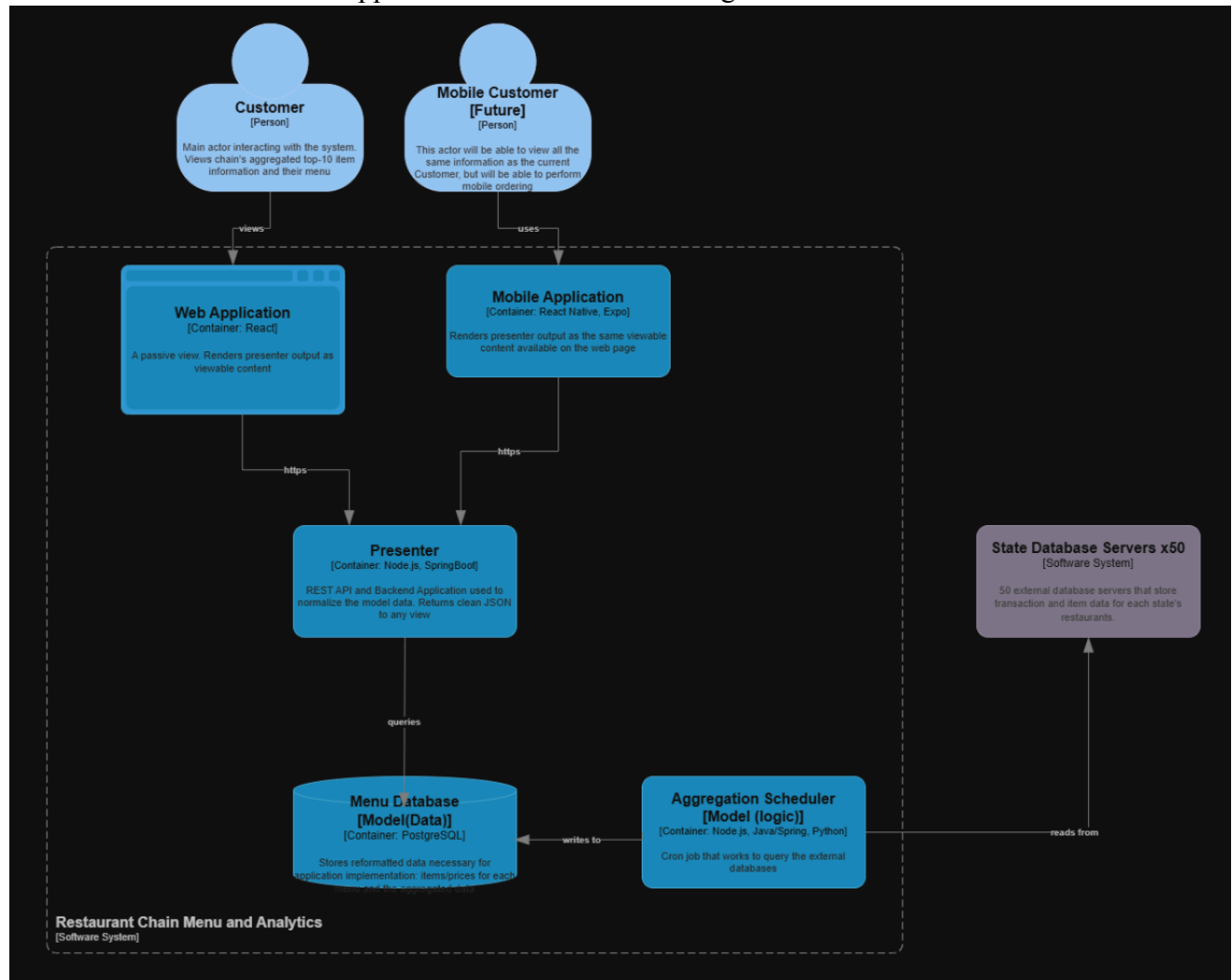


Figure 2. Container Diagram - zooms into the system boundary and maps directly to the MVP layers. At the View layer, the Web Application (React) and Mobile Application (React Native/Expo) both sit above the Presenter. The Customer interacts with the Web App and the Mobile Customer [Future] uses the Mobile App, and both talk to the Presenter over HTTPS.

The Presenter (Node.js/Spring Boot) is the REST API and backend application that normalizes model data and returns clean JSON to whichever View is calling it. Below the Presenter are two Model containers: the Aggregation Scheduler [Model (logic)] container - a cron job running on Node.js, Java/Spring, or Python that reads from the external State Database Servers x50 - and the Menu Database [Model (Data)], a PostgreSQL cylinder that stores the reformatted menu items, prices, and aggregated top-10 data. The Aggregation Scheduler writes to the Menu Database, and the Presenter queries it. The 50 state servers sit outside the system boundary as an external software system, connected only to the Aggregation Scheduler.

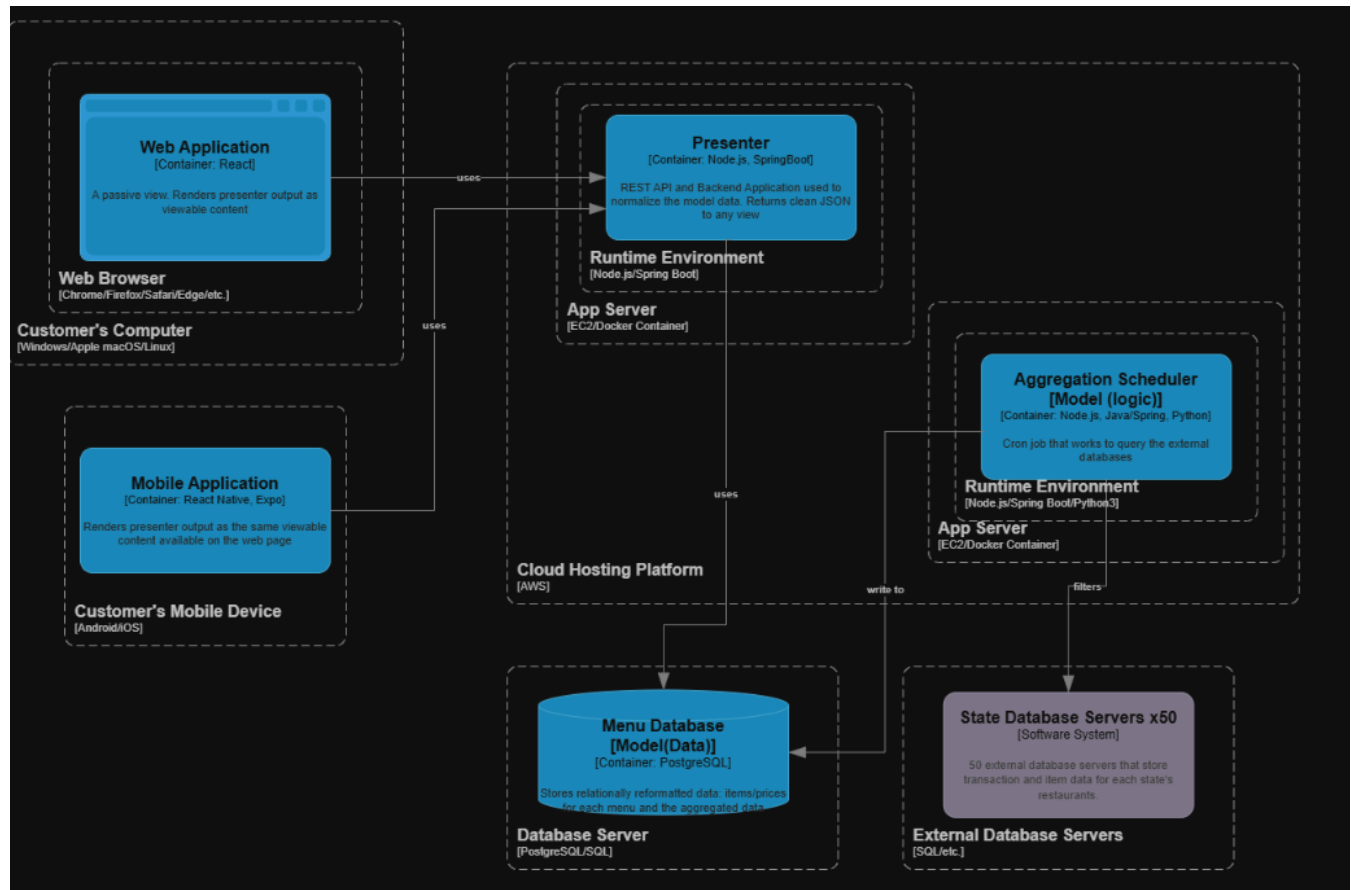


Figure 3. Deployment Diagram - The deployment diagram maps each container to its physical execution environment across four deployment node categories. On the client side, the Web Application (React) runs inside a Web Browser node nested within the Customer's Computer (Windows/macOS/Linux). The Mobile Application (React Native/Expo) runs inside the Customer's Mobile Device (Android/iOS). Both client nodes connect via "uses" relationships to the Presenter, which lives inside an App Server (EC2/Docker Container) within the Cloud Hosting Platform (AWS). The Presenter's App Server is enclosed in a Runtime Environment (Node.js/Spring Boot). The Aggregation Scheduler lives in its own separate App Server (EC2/Docker Container) inside AWS, in its own Runtime Environment (Node.js/Spring Boot/Python3). The Menu Database (PostgreSQL) lives in its own Database Server (PostgreSQL/SQL) deployment node. The Aggregation Scheduler writes to the Menu Database and separately filters and reads from the State Database Servers ×50, which sit in their own External Database Servers (SQL/etc.) boundary node outside the cloud.